ЛАБОРАТОРНА РОБОТА №1

СИСТЕМНЕ ПРОГРАМУВАННЯ

ПОТОКИ. КЛАС THREAD. ПРІОРИТЕТИ. ФОНОВІ ПОТОКИ

Заяць Діани Юріївни

Хід виконання роботи:

1. Створення двох потоків (вивід чисел 1..40 і літер A..Z із затримками)

У першому завданні було реалізовано два окремі класи потоків: NumThread і LetterThread. Клас NumThread відповідає за виконання потоку, який виводить у консоль числа від 1 до 40 із затримкою 200 мс між ітераціями, а клас LetterThread - за потік, який виводить літери від A до Z із затримкою 300 мс. В обох класах під час створення потоку задається Thread.Name, а в основному методі виконання використовується Thread.Sleep(...) для реалізації пауз. Керування запуском і завершенням потоків винесено у Program.cs: у Main створюються об’єкти обох класів, викликається Start() для паралельного старту, після чого головний потік виконує Join() для кожного, щоб дочекатися їх повного завершення. Після цього виводиться повідомлення про завершення завдання, що підтверджує коректне завершення обох потоків.

```
using System;
using System.Threading;

namespace Lab1.Task1
{
    2 references
    public class LetterThread
    {
        private readonly Thread _thread;

        1 reference
        public LetterThread()
        {
            _thread = new Thread(Run)
            {
                Name = "LetterThread"
            };
        }

        1 reference
        public void Start()
        {
            _thread.Start();
        }

        1 reference
        public void Join()
        {
            _thread.Join();
        }

        1 reference
        private void Run()
        {
            for (char c = 'A'; c <= 'Z'; c++)
            {
                Console.WriteLine($"[{Thread.CurrentThread.Name}] {c}");
                Thread.Sleep(300);  // 300 мс
            }
        }
    }
}
```

Рис. 1.1 – Код класу LetterThread (цикл A..Z, Thread.Name, Sleep(300))

```
using System;
using System.Threading;

namespace Lab1.Task1
{
    2 references
    public class NumThread
    {
        private readonly Thread _thread;

        1 reference
        public NumThread()
        {
            _thread = new Thread(Run)
            {
                Name = "NumThread"  // для виводу в консоль
            };
        }

        1 reference
        public void Start()
        {
            _thread.Start();
        }

        1 reference
        public void Join()
        {
            _thread.Join();
        }

        1 reference
        private void Run()
        {
            for (int i = 1; i <= 40; i++)
            {
                Console.WriteLine($"[{Thread.CurrentThread.Name}] {i}");
                Thread.Sleep(200);  // 200 мс між виводами
            }
        }
    }
}
```

Рис. 1.2 – Код класу NumThread (цикл 1..40, Thread.Name, Sleep(200))

```
using System;

namespace Lab1.Task1
{
    0 references
    internal class Program
    {
        0 references
        static void Main(string[] args)
        {
            var numThread = new NumThread();
            var letterThread = new LetterThread();

            numThread.Start();
            letterThread.Start();

            numThread.Join();   // чекаємо обидва
            letterThread.Join();

            Console.WriteLine("Task 1 done");
        }
    }
}
```

Рис. 1.3 – Код Program.cs для завдання 1 (Start() для обох, Join() для обох, повідомлення про завершення)

```
C:\Users\Lenovo Legion\OneD
[NumThread] 12
[NumThread] 13
[LetterThread] I
[NumThread] 14
[LetterThread] J
[NumThread] 15
[LetterThread] K
[NumThread] 16
[NumThread] 17
[LetterThread] L
[NumThread] 18
[LetterThread] M
[NumThread] 19
[NumThread] 20
```

Рис. 1.4 – Результат виконання завдання 1

Поясніть, як працює багатозадачність у межах одного процесу:

У межах одного процесу багатозадачність реалізується через паралельне (або квазіпаралельне) виконання кількох потоків. Кожен потік має власний стек і контекст виконання, але всі потоки одного процесу спільно використовують пам'ять процесу (heap), глобальні дані та ресурси. Планувальник операційної системи періодично розподіляє процесорний час між потоками: видає потоку квант часу, після чого може виконати перемикання контексту на інший потік. На багатоядерних процесорах потоки можуть реально виконуватись одночасно на різних ядрах; на одному ядрі "одночасність" досягається швидким чергуванням. У результаті вивід у консоль виглядає перемішаним, що демонструє роботу планувальника та паралельність виконання потоків.

2. Два foreground-потоки та один background-потік (нескінченний цикл). Показати різницю їх поведінки.

У другому завданні було реалізовано два класи, які демонструють відмінність foreground і background потоків. Для foreground-потоків використано клас CounterThread, який запускає потік-лічильник до заданого значення N (із періодичним виводом у консоль і затримкою Thread.Sleep(...)) та після завершення коректно завершується. Для фонового потоку створено клас BackgroundThread, у якому потік позначено як фоновий через IsBackground = true і реалізовано нескінченний цикл while(true) з періодичним повідомленням у консоль. У Program.cs запускаються всі три потоки, але очікування Join() застосовується тільки для двох foreground-лічильників - це важливо, тому що фонова нитка працює нескінченно і її Join() заблокував би програму назавжди. Після завершення двох foreground-потоків програма виводить пояснення та очікує натискання Enter; після завершення процесу фоновий потік припиняється автоматично, оскільки background-потоки не утримують процес від завершення.

```
using System;
using System.Threading;

namespace Lab1.Task2
{
    3 references
    public class CounterThread
    {
        private readonly Thread _thread;
        private readonly int _countTo;
        private readonly string _threadName;

        2 references
        public CounterThread(string name, int countTo)
        {
            _threadName = name;
            _countTo = countTo;
            _thread = new Thread(Run)
            {
                Name = name,
                IsBackground = false  // foreground — процес чекає завершення
            };
        }

        2 references
        public void Start()
        {
            _thread.Start();
        }

        2 references
        public void Join()
        {
            _thread.Join();
        }

        1 reference
        private void Run()
        {
            for (int i = 0; i <= _countTo; i++)
            {
                Console.WriteLine($"[{Thread.CurrentThread.Name}] Count: {i}");
                Thread.Sleep(100);
            }
            Console.WriteLine($"[{Thread.CurrentThread.Name}] Finished. Counted to {_countTo}");
        }
    }
}
```

Рис. 2.1 – Код класу CounterThread (foreground-лічильник: цикл до N, Sleep, вивід у консоль)

```
using System;
using System.Threading;

namespace Lab1.Task2
{
    2 references
    public class BackgroundThread
    {
        private readonly Thread _thread;
        private static int _iterationCount = 0;

        1 reference
        public BackgroundThread()
        {
            _thread = new Thread(Run)
            {
                Name = "BackgroundThread",
                IsBackground = true  // зупиниться разом із процесом
            };
        }

        1 reference
        public void Start()
        {
            _thread.Start();
        }

        1 reference
        private void Run()
        {
            while (true)  // нескінченний цикл
            {
                _iterationCount++;
                Console.WriteLine($"[{Thread.CurrentThread.Name}] Background message (iteration {_iterationCount})");
                Thread.Sleep(500);
            }
        }
    }
}
```

Рис. 2.2 – Код класу BackgroundThread (IsBackground=true, while(true), Sleep, вивід у консоль)

```
using System;

namespace Lab1.Task2
{
    0 references
    internal class Program
    {
        0 references
        static void Main(string[] args)
        {
            const int N = 10;

            var counter1 = new CounterThread("CounterThread-1", N);
            var counter2 = new CounterThread("CounterThread-2", N);
            var background = new BackgroundThread();

            counter1.Start();
            counter2.Start();
            background.Start();

            counter1.Join();
            counter2.Join();  // тільки foreground – background не чекаємо

            Console.WriteLine();
            Console.WriteLine("Foreground threads finished. Background thread has IsBackground=true,");
            Console.WriteLine("so it will be stopped when the process exits (e.g. after pressing Enter).");
            Console.WriteLine("Press Enter to exit...");
            Console.ReadLine();  // без Enter background крутився б далі
        }
    }
}
```

Рис. 2.3 – Код Program.cs для завдання 2 (Start() трьох потоків, Join() тільки для двох CounterThread, ReadLine перед виходом)

```
[CounterThread-2] Count: 4
[BackgroundThread] Background message (iteration 2)
[CounterThread-2] Count: 5
[CounterThread-1] Count: 5
[CounterThread-1] Count: 6
[CounterThread-2] Count: 6
[CounterThread-2] Count: 7
[CounterThread-1] Count: 7
[CounterThread-1] Count: 8
[CounterThread-2] Count: 8
[CounterThread-2] Count: 9
[CounterThread-1] Count: 9
[BackgroundThread] Background message (iteration 3)
[CounterThread-2] Count: 10
[CounterThread-1] Count: 10
[CounterThread-1] Finished. Counted to 10
[CounterThread-2] Finished. Counted to 10

Foreground threads finished. Background thread has IsBackground=true,
so it will be stopped when the process exits (e.g. after pressing Enter).
Press Enter to exit...
[BackgroundThread] Background message (iteration 4)
[BackgroundThread] Background message (iteration 5)
[BackgroundThread] Background message (iteration 6)
[BackgroundThread] Background message (iteration 7)
[BackgroundThread] Background message (iteration 8)
[BackgroundThread] Background message (iteration 9)
```

Рис. 2.4 – Результат виконання завдання 2 (паралельний вивід двох лічильників і фонового потоку, повідомлення про Enter/очікування)

Що відбудеться з фоновим потоком після завершення пріоритетних потоків?

Фоновий потік (IsBackground = true) не утримує процес від завершення. Тому коли всі foreground-потоки (звичайні/пріоритетні) завершаться і головний потік програми дійде до завершення, процес завершиться, а всі background-потоки будуть примусово зупинені операційною системою разом із завершенням процесу. Це означає, що фоновий потік може не “докрутити” свій код до логічного завершення - він просто припиняється в момент завершення програми.

3. Потоки з різними пріоритетами, синхронний старт через ManualResetEvent, таблиця результатів і % CPU(варіант 3)

У третьому завданні (варіант 3) було реалізовано клас WorkerThread, який інкапсулює робочий потік із заданими Thread.Name та Thread.Priority, а також зберігає результати виконання: кількість ітерацій та час роботи. Кожен WorkerThread виконує однакову задачу - підрахунок до 100 000 000, але відрізняється пріоритетом: Lowest, AboveNormal, BelowNormal, Highest. Щоб запуск усіх потоків був максимально одночасним, у реалізації використано ManualResetEvent: у методі виконання потоку спочатку викликається WaitOne(), і потік “стоїть” у очікуванні сигналу, а в Program.cs після створення і старту всіх потоків викликається Set(), що розблоковує їх одночасно. Для вимірювання тривалості виконання в кожному потоці використано Stopwatch, а після завершення всіх потоків через Join() у Program.cs формується підсумкова таблиця з Name, Priority, Iterations, ElapsedTime(ms). Далі розраховується відсотковий розподіл CPU за формулою score = 1/elapsed, percent = (score/sumScores)*100, і значення виводяться з двома знаками після коми, що дозволяє порівняти відносний вплив пріоритетів на швидкість виконання однакової роботи.

```
using System;
using System.Diagnostics;
using System.Threading;

namespace Lab1.Task3
{

    public class WorkerThread
    {

        public string Name { get; }

        public ThreadPriority Priority { get; }

        public long Iterations { get; private set; }

        public long ElapsedMs { get; private set; }

        public Thread Thread { get; }

        private readonly ManualResetEvent _startSignal;  // один на всіх — синхронний старт
        private const long Limit = 100_000_000;


        public WorkerThread(string name, ThreadPriority priority, ManualResetEvent startSignal)
        {
            Name = name;
            Priority = priority;
            _startSignal = startSignal;
            Thread = new Thread(Run)
            {
                Name = name,
                Priority = priority
            };
        }


        public void Start()
        {
            Thread.Start();
        }

        1 reference
        public void Join()
        {
            Thread.Join();
        }

        1 reference
        private void Run()
        {
            _startSignal.WaitOne();  // чекаємо сигнал — стартуємо разом

            var sw = Stopwatch.StartNew();
            long count = 0;
            for (long i = 0; i < Limit; i++)
            {
                count++;
            }
            sw.Stop();

            Iterations = count;
            ElapsedMs = sw.ElapsedMilliseconds;
        }
    }
}
```

Рис. 3.1 – Код класу WorkerThread (WaitOne(), Stopwatch, цикл до 100 000 000, фіксація Elapsed/Iterations)

```
using System;
using System.Threading;

namespace Lab1.Task3
{
    0 references
    internal class Program
    {
        0 references
        static void Main(string[] args)
        {
            var startSignal = new ManualResetEvent(false);  // спочатку не сигналізовано

            var workers = new[]
            {
                new WorkerThread("T-Lowest", ThreadPriority.Lowest, startSignal),
                new WorkerThread("T-Above", ThreadPriority.AboveNormal, startSignal),
                new WorkerThread("T-Below", ThreadPriority.BelowNormal, startSignal),
                new WorkerThread("T-Highest", ThreadPriority.Highest, startSignal)
            };

            foreach (var w in workers)
                w.Start();

            startSignal.Set();  // усі виходять з WaitOne і починають

            foreach (var w in workers)
                w.Join();

            Console.WriteLine();
            Console.WriteLine("| {0,-10} | {1,-12} | {2,12} | {3,14} |", "Name", "Priority", "Iterations", "ElapsedTime(ms)");
            Console.WriteLine("|------------|--------------|--------------|----------------|");

            foreach (var w in workers)
            {
                Console.WriteLine("| {0,-10} | {1,-12} | {2,12} | {3,14} |",
                    w.Name, w.Priority, w.Iterations, w.ElapsedMs);
            }

            double sumScores = 0;
            foreach (var w in workers)
                sumScores += 1.0 / w.ElapsedMs;  // score = 1/час

            Console.WriteLine();
            Console.WriteLine("CPU time distribution (%):");
            foreach (var w in workers)
            {
                double score = 1.0 / w.ElapsedMs;
                double percent = (score / sumScores) * 100;  // % від загального score
                Console.WriteLine("  {0}: {1:F2}%", w.Name, percent);
            }
        }
    }
}
```

Рис. 3.2 – Код Program.cs для завдання 3 (ManualResetEvent(false), створення 4 WorkerThread з пріоритетами варіанта 3, Start(), Set(), Join(), формування таблиці та %)

```
| Name       | Priority     |   Iterations | ElapsedTime(ms) |
|------------|--------------|--------------|----------------|
| T-Lowest   | Lowest       |    100000000 |            144 |
| T-Above    | AboveNormal  |    100000000 |            150 |
| T-Below    | BelowNormal  |    100000000 |            137 |
| T-Highest  | Highest      |    100000000 |            147 |

CPU time distribution (%):
  T-Lowest: 25,06%
  T-Above: 24,06%
  T-Below: 26,34%
  T-Highest: 24,55%
```

Рис. 3.2 – Результат виконання завдання 3 (таблиця Name/Priority/Iterations/ElapsedTime(ms) і блок CPU time distribution (%))

Чи завжди високопріоритетний потік отримує найбільше часу ЦП? Які фактори можуть впливати на планування?

Ні, не завжди. Вищий пріоритет лише підвищує ймовірність, що потік отримає процесорний час частіше, але реальний розподіл залежить від багатьох факторів:

- Навантаження системи та інші процеси/потоки: якщо паралельно працюють інші програми або потоки з високими пріоритетами, вони конкурують за CPU.
- Кількість ядер: на багатоядерній системі потоки можуть виконуватись одночасно, і ефект пріоритетів може бути менш помітним.
- Стан потоку (готовий/заблокований): якщо потік чекає I/O, Sleep(), WaitOne(), lock або іншу синхронізацію, він тимчасово не претендує на CPU незалежно від пріоритету.
- Алгоритми планувальника ОС: планувальник може використовувати внутрішні політики “справедливості”, динамічні корекції пріоритетів, енергозбереження тощо.
- Особливості виконання коду: різниця в інструкціях, оптимізаціях JIT, кешах CPU та частота звернень до пам’яті можуть впливати на фактичну швидкість виконання, навіть якщо задача “однакова”.

Отже, високопріоритетний потік зазвичай має перевагу, але не має гарантії отримати найбільшу частку процесорного часу в кожному запуску.

4. <u>N потоків за вводом користувача, введення пріоритетів, realtime-прогрес, підсумкова таблиця і % CPU.</u>

У четвертому завданні було реалізовано консольний застосунок, у якому користувач задає кількість потоків N (у межах 1..12) та пріоритет для кожного потоку, після чого всі потоки стартують синхронно за тим самим принципом, що й у завданні 3, через ManualResetEvent. Для виконання роботи збережено підхід із WorkerThread: кожен потік виконує підрахунок до заданого ліміту та паралельно оновлює поточний лічильник прогресу, який потрібен для відображення “живого” стану виконання. Окремо в Program.cs створено потік моніторингу прогресу (progress monitor), який працює у фоні (IsBackground = true) і приблизно кожні 500 мс виводить по кожному воркеру його ім’я, пріоритет, стан (Running/Stopped) і поточне значення лічильника. Щоб читання поточного значення було коректним при одночасному оновленні з іншого потоку, для роботи з лічильником використано атомарні операції (Interlocked). Після завершення всіх робочих потоків у Program.cs друкується фінальна таблиця результатів та відсотковий розподіл CPU за тією ж формулою, що й у завданні 3 (score = 1/elapsed, percent = score/sumScores*100), що дозволяє порівняти, як задані користувачем пріоритети впливають на результат.

```
using System;
using System.Diagnostics;
using System.Threading;

namespace Lab1.Task4
{
    3 references
    public class WorkerThread
    {
        4 references
        public string Name { get; }
        3 references
        public ThreadPriority Priority { get; }
        1 reference
        public long Iterations => _currentCount;
        4 references
        public long ElapsedMs { get; private set; }
        6 references
        public Thread Thread { get; }
        0 references
        public bool IsRunning => Thread.IsAlive;
        1 reference
        public string State => Thread.IsAlive ? "Running" : "Stopped";

        private long _currentCount;
        private readonly ManualResetEvent _startSignal;
        private const long Limit = 100_000_000;

        1 reference
        public WorkerThread(string name, ThreadPriority priority, ManualResetEvent startSignal)
        {
            Name = name;
            Priority = priority;
            _startSignal = startSignal;
            Thread = new Thread(Run)
            {
                Name = name,
                Priority = priority
            };
        }

        1 reference
        public void Start()
        {
            Thread.Start();
        }

        1 reference
        public void Join()
        {
            Thread.Join();
        }
```

```
        1 reference
        public long GetCurrentCount()  // ProgressMonitor читає з іншого потоку
        {
            return Interlocked.Read(ref _currentCount);
        }

        1 reference
        private void Run()
        {
            _startSignal.WaitOne();  // старт разом з іншими

            var sw = Stopwatch.StartNew();
            for (long i = 0; i < Limit; i++)
            {
                Interlocked.Exchange(ref _currentCount, i + 1);  // атомарно для читання з ProgressMonitor
            }
            sw.Stop();
            ElapsedMs = sw.ElapsedMilliseconds;
        }
    }
}
```

Рис. 4.1 – Код WorkerThread для завдання 4 (оновлення поточного лічильника прогресу та метод/властивість для його читання)

```
using System;
using System.Threading;

namespace Lab1.Task4
{
    0 references
    internal class Program
    {
        private const long Limit = 100_000_000;

        1 reference
        static ThreadPriority ParsePriority(string input)  // рядок -> пріоритет
        {
            switch (input?.ToLowerInvariant())
            {
                case "highest": return ThreadPriority.Highest;
                case "abovenormal": return ThreadPriority.AboveNormal;
                case "normal": return ThreadPriority.Normal;
                case "belownormal": return ThreadPriority.BelowNormal;
                case "lowest": return ThreadPriority.Lowest;
                default: return ThreadPriority.Normal;
            }
        }

        0 references
        static void Main(string[] args)
        {
            int n;
            do
            {
                Console.Write("Enter N (1..12): ");
                string line = Console.ReadLine();
                if (!int.TryParse(line, out n) || n < 1 || n > 12)  // перевірка діапазону
                {
                    Console.WriteLine("Invalid. Enter a number from 1 to 12.");
                    continue;
                }
                break;
            } while (true);

            var workers = new WorkerThread[n];
            var startSignal = new ManualResetEvent(false);

            Console.WriteLine("Enter priority for each thread: Highest, AboveNormal, Normal, BelowNormal, Lowest");
            for (int i = 0; i < n; i++)
            {
                Console.Write($"Thread {i + 1} priority: ");
                string prioStr = Console.ReadLine();
                var prio = ParsePriority(prioStr);
                string name = $"Worker-{i + 1}";
                workers[i] = new WorkerThread(name, prio, startSignal);
            }
```

```
            var progressThread = new Thread(() =>  // realtime прогрес кожні 500 мс
            {
                while (true)
                {
                    Thread.Sleep(500);
                    bool anyAlive = false;
                    Console.WriteLine();
                    foreach (var w in workers)
                    {
                        if (w.Thread.IsAlive) anyAlive = true;
                        Console.WriteLine("  {0,-12} Priority={1,-12} State={2,-8} CurrentCount={3}",
                            w.Name, w.Priority, w.State, w.GetCurrentCount());
                    }
                    if (!anyAlive) break;
                }
            })
            {
                IsBackground = true,
                Name = "ProgressMonitor"
            };

            foreach (var w in workers)
                w.Start();

            progressThread.Start();
            startSignal.Set();  // усі воркери стартують одночасно

            foreach (var w in workers)
                w.Join();

            Thread.Sleep(600);  // щоб останній вивід прогресу встиг
            Console.WriteLine();
            Console.WriteLine("All threads finished.");
            Console.WriteLine();
            Console.WriteLine("| {0,-12} | {1,-12} | {2,12} | {3,14} |", "Name", "Priority", "Iterations", "ElapsedTime(ms)");
            Console.WriteLine("|--------------|--------------|--------------|----------------|");

            foreach (var w in workers)
            {
                Console.WriteLine("| {0,-12} | {1,-12} | {2,12} | {3,14} |",
                    w.Name, w.Priority, w.Iterations, w.ElapsedMs);
            }

            double sumScores = 0;
            foreach (var w in workers)
                sumScores += 1.0 / w.ElapsedMs;

            Console.WriteLine();
            Console.WriteLine("CPU time distribution (%):");
            foreach (var w in workers)
            {
                double score = 1.0 / w.ElapsedMs;
                double percent = (score / sumScores) * 100;
                Console.WriteLine("  {0}: {1:F2}%", w.Name, percent);
            }
        }
```

Рис. 4.2 – Код Program.cs для завдання 4 (ввід N і пріоритетів, створення воркерів, запуск progress monitor, Set(), Join(), фінальна таблиця та %)

```
Enter N (1..12): 5
Enter priority for each thread: Highest, AboveNormal, Normal, BelowNormal, Lowest
Thread 1 priority: Highest
Thread 2 priority: Normal
Thread 3 priority: AboveNormal
Thread 4 priority: BelowNormal
Thread 5 priority: Lowest
```

Рис. 4.3 – Ввід даних у завданні 4 (екран консолі з введенням N та пріоритетів)

```
  Worker-1     Priority=Highest      State=Running  CurrentCount=53604713
  Worker-2     Priority=Normal       State=Running  CurrentCount=51349537
  Worker-3     Priority=AboveNormal  State=Running  CurrentCount=50864505
  Worker-4     Priority=BelowNormal  State=Running  CurrentCount=52303856
  Worker-5     Priority=Lowest       State=Running  CurrentCount=51027244

  Worker-1     Priority=Highest      State=Stopped  CurrentCount=100000000
  Worker-2     Priority=Normal       State=Stopped  CurrentCount=100000000
  Worker-3     Priority=AboveNormal  State=Stopped  CurrentCount=100000000
  Worker-4     Priority=BelowNormal  State=Stopped  CurrentCount=100000000
  Worker-5     Priority=Lowest       State=Stopped  CurrentCount=100000000

All threads finished.

| Name         | Priority     |   Iterations | ElapsedTime(ms) |
|--------------|--------------|--------------|----------------|
| Worker-1     | Highest      |    100000000 |            929 |
| Worker-2     | Normal       |    100000000 |            947 |
| Worker-3     | AboveNormal  |    100000000 |            975 |
| Worker-4     | BelowNormal  |    100000000 |           1012 |
| Worker-5     | Lowest       |    100000000 |            958 |

CPU time distribution (%):
  Worker-1: 20,74%
  Worker-2: 20,35%
  Worker-3: 19,76%
  Worker-4: 19,04%
  Worker-5: 20,11%
```

Рис. 4.4 – Realtime-прогрес і результат завдання 4

Висновок:

У ході виконання лабораторної роботи було реалізовано набір консольних програм на C#, що демонструють практичну роботу з потоками через System.Threading.Thread. Було відпрацьовано створення та запуск потоків, задання імен потоків для ідентифікації у виводі, керування часом виконання через Thread.Sleep(), а також коректне очікування завершення потоків методом Join(). На прикладі foreground і background потоків підтверджено, що фонові потоки не утримують процес від завершення і припиняються разом із завершенням програми. Для завдань із пріоритетами реалізовано керування Thread.Priority та синхронний старт через ManualResetEvent, що дозволило порівнювати виконання потоків у максимально однакових умовах. Додатково було реалізовано моніторинг прогресу виконання в реальному часі та формування підсумкової таблиці з оцінкою відсоткового розподілу “процесорного часу” між потоками, що дало практичне розуміння впливу пріоритетів та факторів планування ОС на поведінку багатопотокових програм.